

E.P. Interiors — Studio Suite: Full Spec

Context

This POC (Express server + `/api/search` endpoint + test page) proved two things while working with Claude in a separate chat conversation:

1. Real product images load fine on a real server (they do NOT load inside Claude's artifact sandbox — that was the whole reason this project exists).
2. Calling Claude's API server-side, with the key held only on the backend, works and is affordable (~\$0.15-0.20 per web-search-backed call at time of testing — plan for real usage volume, not "a few cents").

The starting point for the real build is a working prototype called the **E.P. Interiors Studio Suite** — a set of 8 connected tools, currently built as a single-file HTML/JS artifact inside Claude's chat interface (client-side only, browser storage, no real backend). That artifact works functionally but has three hard limitations we're now fixing by building a real app:

- No live product image loading (Claude artifact sandbox restriction)
- No real persistence (browser-only storage, not portable, not backed up)
- No secure place to hold API keys or real integrations (QuickBooks, Google Sheets)

The goal: rebuild the same 8 tools as a real web app with a real backend, keeping all the business logic and visual design, replacing only the plumbing.

Brand / Visual Design (keep exactly)

- **Studio name:** E.P. Interiors — "Thoughtful Spaces, Intentional Living"
- **Colors:** Cocoa (■ #584435) (text), Citron (■ #9E8E3D) (primary/buttons), Moss (■ #666644) (muted/secondary), Sky (■ #CAD4D0) (borders), Cloud (■ #F0EDE8) (light backgrounds/panels), Berry (■ #661921) (danger/alerts)
- **Fonts:** Libre Baskerville (serif, for headings — weights 400/700 only), Montserrat (sans, body/UI), IBM Plex Mono (data/numeric fields, tags, labels)
- **Tone:** editorial, warm, professional — not corporate SaaS-generic. White backgrounds, black headings, citron buttons, cocoa body text.
- There's a real brand crest (monogram) image that should appear in the header — ask the user for it, they have it as a PNG.

The 8 Tools (tabs in one app)

1. Schedule Builder — the core/hub

The master FF&E (Furniture, Fixtures & Equipment) product list for a project, organized by Room.

Each **line item** needs these fields:

- `category` (Upholstery / Case Goods / Lighting / Rugs / Accessories / Window Treatments / Other)
- `item` (name), `vendor`, `sku`, `finish`, `dims`, `qty`
- **Pricing (the important part — mirrors a real pricing model the client already uses in a separate Google Sheet):**
 - `tradeCost` (what the designer pays), `markupPct` OR `markupAmt` (whichever is entered drives the other, and drives Client Price forward — this bidirectional calc is important, already implemented once, ask the previous artifact code for reference)
 - `clientPrice` = `tradeCost` + `markupAmt` (computed)
 - `tradeTaxPct` / `tradeTaxAmt`, `clientTaxPct` / `clientTaxAmt`
 - `shippingCost`, `shippingMarkupPct` / `shippingMarkupAmt` (same forward-calc pattern), `clientShippingPrice`
 - `receivingCost`, `receivingMarkupPct`, `clientReceivingCost`
 - `totalCostAllIn`, `totalClientAllIn`, `profitAmt`, `profitMarginPct` (all computed)
- `leadTime`, `status` — one of: **Considering, Proposed, Approved, Ordered, Order Confirmed, Backordered, Shipped, Received, Installed, Returned**
- `imageUrl`, `sourceUrl` (product link)
- `includeOnInvoice` (boolean checkbox — separate from status)
- `invoicedId` (set once invoiced), `poId` (set once put on a purchase order)
- `notes`

Project switcher: the app needs to support **multiple projects**, each with its own full set of rooms/items/invoices/POs. A project has at minimum: name, and probably later client contact info (see Cover Sheet below).

Two ways to add items:

- AI-generate from pasted messy text/notes (call Claude, ask it to extract structured line items)
- "Quick add from a link" — paste a product URL, Claude searches and fills in the fields

2. Sourcing Specialist

Natural-language product search ("30-32W swivel glider under \$3K, lead time under 8 weeks") → calls Claude with the `web_search` tool → returns a shortlist of real candidate products (name, vendor, price, dims, lead time, url, image, one-line fit reasoning).

Vendor tiering (important, already spec'd with a real client vendor list): the client has a real list of ~387 vendors tiered 1/2/3 (Tier 1 = budget/marketplace like Amazon/Target/Wayfair, Tier 2 = mid-range, Tier 3 = designer/luxury/trade). A tier filter should constrain the AI search to only suggest vendors from that tier. **Ask the user for this vendor list** (they have it, gave it to Claude in the other chat as a big pasted table with columns: Vendor, Tier, Category, USA Shipping Status, Trade Only/Trade Leaning, Designer Favorite, Tier Notes).

Each result should have a "+ **Add to Schedule**" button that pushes it directly into the current project's Schedule (as a new line item, room defaults to "General").

3. Tear Sheet Builder

Generates a branded, printable client-facing product sheet — pulls an item either (a) directly from Schedule (preferred — no AI call needed, just render the existing data) or (b) from a pasted URL/note (AI call). Shows: item name, image, room, install location, dimensions, material/finish, quantity, investment (client price), lead time, designer notes (editable), a client-approval signature line. Should support a **Print/Export to PDF** action — this is one of the few places a "real embedded photo in a real PDF" matters most, since this document may get printed or emailed to clients.

4. Quality Checker (Pre-Order Verifier)

Paste "what was specified" and "what's in the cart/PO" as two text blocks; Claude compares them field by field (product, finish, size, qty), runs a standard checklist (ship-to correct, PO# present, trade discount applied, in-stock confirmed), and gives a pass/warn/fail flag with a plain-language summary before the designer places a real order.

5. Order Tracker

Pulls every Schedule item that has a `poId` set (i.e., is on a Purchase Order). Two ways to update status: (a) manually via a dropdown per item, or (b) paste vendor emails/tracking notices as text; Claude matches them to tracked items and proposes status updates + flags anything at risk of a deadline slip + drafts a client update email. **Status changes must write back to the same item in Schedule** — this is not a separate copy of the data, it's the same underlying item.

6. Invoice Creator

Internal staging tool (not client-facing — the client only ever sees a real QuickBooks

invoice). Pulls every Schedule item with `includeOnInvoice = true` and no `invoicedId` yet. Lets the user fine-tune which of those go into this specific invoice batch, totals it (client price, client shipping, client tax → grand total), and on "Create Invoice" stamps an invoice ID onto those items in Schedule and logs the invoice (id, date, total, item count) to invoice history for that project.

7. Purchase Orders

Pulls every Schedule item that has been **invoiced** but not yet **PO'd**, and groups them **by vendor automatically**. Each vendor group can be turned into its own PO — uses the designer's **cost side** (trade cost, trade tax, shipping, receiving), not client pricing, since this is what's owed to the vendor. Stamps a PO ID onto those items and logs PO history (id, vendor, date, total, item count).

8. Income & Expense

Rolls up Invoices (income) + Purchase Orders (expense) automatically per project into Total Income / Total Expenses / Net / Margin. Also supports manual entries for things that don't flow through Invoices/POs (design fees, retainers, misc costs).

Cross-Tool Data Flow (critical — this is the whole point of doing this "for real")

Items are not separate records duplicated across tools — **one item, one identity**, referenced everywhere:

```
Schedule (source of truth)
  → item checked "Include on Invoice"
  → Invoice Creator pulls it, batches it, stamps invoicedId back onto the Schedule item
  → Purchase Orders pulls anything with invoicedId but no poId, groups by vendor, stamps poId back onto the Schedule item
  → Order Tracker pulls anything with poId, tracks/updates status, writes status back onto the Schedule item
  → Income & Expense reads Invoice history (income) + PO history (expense) for the rollup
```

In the old client-side artifact, this was done via a shared in-memory JS object

`(window.EPSchedule)` bridge with methods like `getAllItems()`, `markInvoiced()`, `markPO()`, `updateItemStatus()`). In the real app, this is just... a real relational database. Items live in

one table, other tables reference them by ID. Much simpler than the workaround we needed client-side.

Data Persistence — Target

Use a **real hosted Postgres database, not SQLite on local disk**. This matters specifically because of how the hosting works: Render's free/cheap web service tiers do **not** have a persistent filesystem — anything written to disk (including a SQLite file) is wiped on every redeploy or restart. A real Postgres instance (Render's own Postgres addon, or a free-tier external one like Supabase/Neon) is required for data to actually survive.

Suggested schema (rough):

- `projects` (id, name, client_name, client_address, client_email, client_phone, created_at)
- `rooms` (id, project_id, name)
- `items` (id, room_id, all the Schedule fields listed above)
- `invoices` (id, project_id, invoice_number, date, total, item_ids[])
- `purchase_orders` (id, project_id, po_number, vendor, date, total, item_ids[])
- `finance_entries` (id, project_id, category, description, type, amount, date)
- `vendors` (id, name, tier, category) — seeded once from the client's real vendor list

API / Backend notes

- The existing `server.js` pattern (Express route holding `ANTHROPIC_API_KEY` server-side, frontend calls our own `/api/...` routes, never the Anthropic API directly) is correct — keep extending it, one route per AI-backed action (generate-schedule, sourcing-search, quality-check, order-tracker-scan, tear-sheet-generate).
- `web_search` tool usage (Sourcing Specialist, Quick-add-from-link, Tear Sheet from URL) — same tool-use pattern as the POC's `/api/search`, just more endpoints.

Explicitly out of scope for now

- **QuickBooks integration** — the client has a separate, existing Google Apps Script + QuickBooks OAuth setup (built in a different conversation) that isn't cleanly callable from outside Google Sheets. Not rebuilding that here yet — Invoice Creator just produces the invoice data; getting it into QuickBooks stays a manual step for now.
- **Google Sheets sync** — same story, manual/out of scope for this build.
- **Auth/login** — single-user for now (the designer herself); ask before adding multi-user accounts.

Immediate next step

Given the current POC only has the one `/api/search` route, the natural first milestone is:

stand up the Postgres database + the Schedule Builder tool end-to-end (create project, add rooms/items by hand and via AI-generate, edit pricing fields with the forward-calculating markup logic, real image display). Get that solid before adding the other 7 tools — it's the hub everything else depends on.